



CNN-Adaptive Sliding Mode Control and Reinforcement Learning for Autonomous Mobile Robot Navigation: A Digital Twin Approach

PGE5 — AI Clinic Research Report

Participating Students	Likhita Yerra Mohamed Oussama Bouriga Ahmed Ben Aissa Abdellahi El Moustapha Thibault Goutorbe
Supervisor	Anuradha Kar
Date of Submission	28 August 2026

aivancity — School of Technology and Management for AI
Paris, France

August 28, 2026

1 Introduction

1.1 Problem Statement

Autonomous mobile robots navigating in unstructured environments must maintain accurate trajectory tracking despite sensor noise, mechanical disturbances, and surface-induced wheel slip. Sliding Mode Control (SMC) is widely adopted in this domain because of its intrinsic robustness to matched disturbances and parameter variations [?, ?]. However, classical SMC uses a single fixed set of gains that cannot simultaneously optimise chattering suppression, tracking accuracy, and disturbance rejection across all operating regimes.

When sensors become noisy, fixed gains produce excessive chattering in the angular velocity command. When an external push is applied or wheels lose traction, the same fixed gains may recover too slowly or too aggressively. This motivates *scenario-aware* control: adapting SMC parameters based on the operating environment rather than using one preset for all conditions.

This project addresses the research question:

Can a learning-based perception and control pipeline improve the adaptability of sliding mode control for a differential-drive mobile robot under heterogeneous uncertainty conditions?

We propose and evaluate:

1. a **CNN-adaptive SMC** framework that classifies the environment from a 64×64 occupancy map and selects hand-tuned SMC parameter presets;
2. a **PPO reinforcement learning agent** that learns continuous SMC gain adaptation from a multi-objective reward signal; and
3. a **3D digital twin** web interface for live simulation, controller comparison, replay, and demonstration.

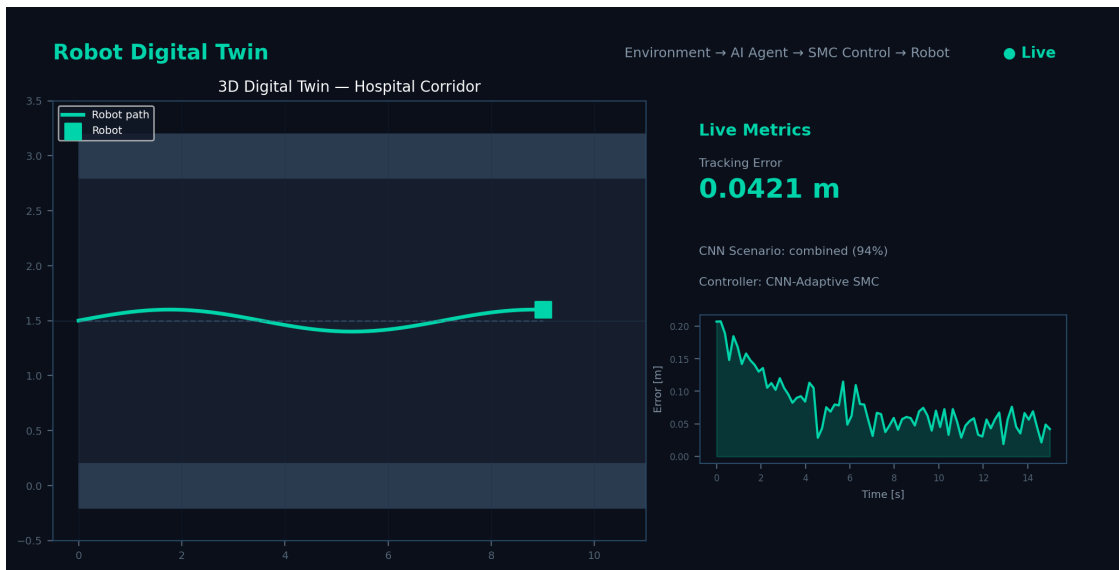


Figure 1: 3D digital twin interface for live robot simulation, metrics dashboard, and controller comparison.

1.2 Literature Review

Sliding Mode Control for Mobile Robots. SMC provides finite-time convergence and robustness to matched uncertainties [?]. Anti-chattering techniques—boundary layers, saturation switching, and higher-order SMC—are well established [?, ?]. Adaptive SMC with Lyapunov-based gain tuning can converge slowly or require tight disturbance bounds [?]. Differential-drive platforms benefit from SMC due to nonholonomic constraints and sensitivity to lateral disturbances.

Learning-Based Gain Adaptation. Fuzzy-SMC hybrids [?] and neural-network augmented controllers [?] tune switching gains automatically. Reinforcement learning combined with SMC has been explored for aerial and manipulation platforms; PPO [?] offers stable policy gradient updates for continuous control. A limitation of pure learning approaches is opacity: resulting gain schedules are difficult to interpret or certify, motivating our use of hand-tuned presets selected by a learned classifier.

CNN-Based Supervisory Control. Supervisory switching architectures that select among pre-designed controllers based on a mode estimate are established in hybrid systems theory [?]. CNN-based perception has transformed autonomous navigation [?, ?], yet direct use of CNN classification for SMC gain scheduling on differential-drive platforms has received comparatively little attention in reproducible student-scale pipelines.

Digital Twins. Digital twin technology provides a virtual replica of a physical system for monitoring, simulation, and decision support. For robotics research, a web-based 3D twin enables live demonstration of control algorithms, side-by-side controller comparison, and stakeholder communication during project defense.

Research Gap. While SMC, CNN-based perception, and RL-based control have each been studied independently, few projects integrate all three within a unified simulation and visualization platform. This work fills that gap by implementing classical SMC, CNN-adaptive SMC, and PPO-adaptive SMC on the same robot model and evaluating them under identical scenarios.

2 Methodology

2.1 Robot and Simulation Model

The robot is modelled as a unicycle differential-drive with state $\mathbf{q} = [x, y, \theta, v, \omega]^\top$. Continuous-time kinematics:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega. \quad (1)$$

Discrete integration uses $\Delta t = 0.01$ s. Physical parameters: wheel base $L = 0.3$ m, wheel radius $r = 0.05$ m. Uncertainty is injected through three mechanisms:

- **Noise:** zero-mean Gaussian noise on pose measurements ($\sigma_{xy} = 0.02$ m, $\sigma_\theta = 0.01$ rad);
- **Disturbance:** single external push of +0.4 m in y at $t = 8$ s;

- **Slip:** 30% velocity reduction during $t \in [10, 14]$ s.

Five scenarios are evaluated: Normal, Noise, Disturbance, Slip, and Combined (all effects present).

2.2 CNN-Adaptive SMC Architecture

Tracking errors are $e_x = x_r - x$, $e_y = y_r - y$, $e_\theta = \theta_r - \theta$. Sliding surfaces:

$$s_i = \dot{e}_i + \lambda_i e_i, \quad i \in \{x, y, \theta\}. \quad (2)$$

The angular velocity command uses hyperbolic-tangent switching to suppress chattering, with exponential smoothing coefficient α_ω .

System Architecture — CNN-Adaptive SMC Pipeline

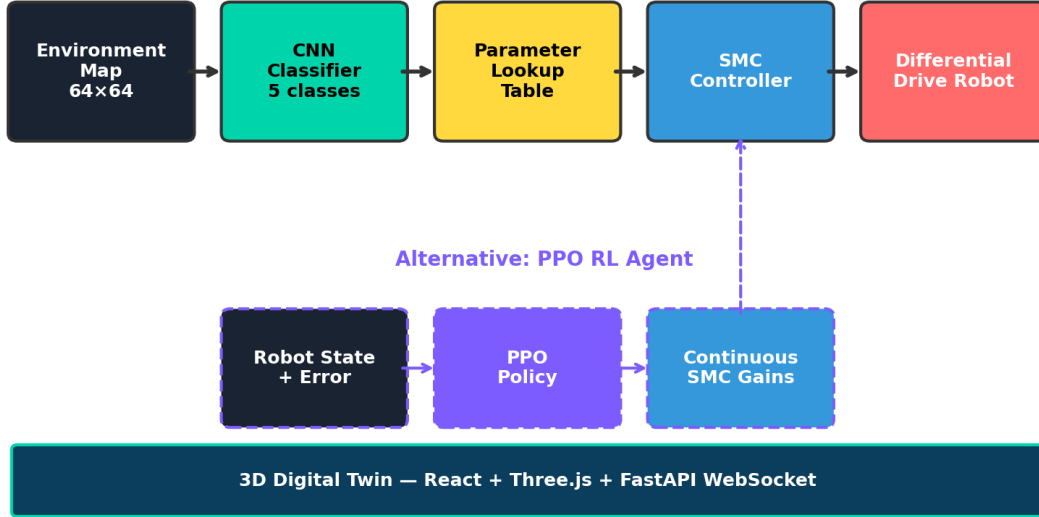


Figure 2: System architecture: CNN classifies a pre-mission map and selects SMC presets; PPO RL provides a continuous alternative; both feed a 3D digital twin.

EnvironmentCNN. Synthetic 64×64 grayscale maps are generated for five classes (Fig. 3). The CNN comprises three convolutional blocks ($16 \rightarrow 32 \rightarrow 64$ filters) with batch normalization, ReLU, max pooling, and a two-layer classifier with dropout ($\approx 534k$ parameters). Training uses Adam ($\text{lr} = 10^{-3}$, batch size 32) with cross-entropy loss.

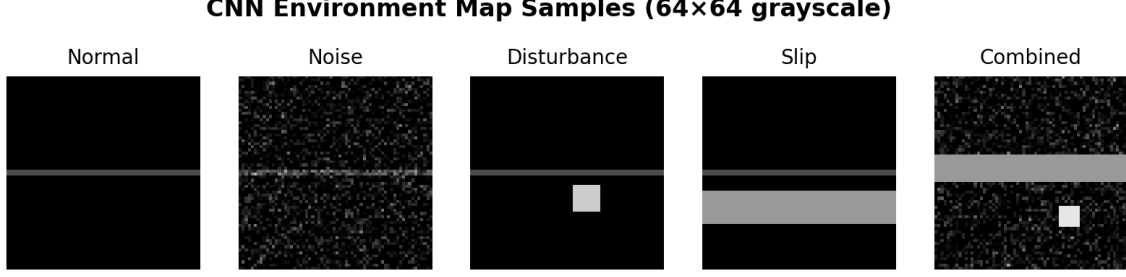


Figure 3: Synthetic environment map samples (one per scenario class) used for CNN training.

At deployment, the CNN classifies a pre-mission map and maps the predicted label to a hand-tuned parameter set $\pi_{\hat{c}} = \{\lambda_x, \lambda_y, \lambda_\theta, k_v, k_\omega, \phi, \alpha_\omega\}$. An exponential filter prevents abrupt gain jumps:

$$\pi_k = (1 - \beta) \pi_{k-1} + \beta \pi_{\hat{c}}, \quad \beta \in (0, 1]. \quad (3)$$

2.3 Reinforcement Learning Agent (PPO)

A Gymnasium environment wraps the SMC simulation with domain randomization. The PPO agent uses an actor-critic architecture with Generalized Advantage Estimation (GAE). The per-step reward balances tracking accuracy, control effort, and chattering:

$$r = -\alpha e^2 - \beta (v^2 + \omega^2) - \gamma |\Delta\omega|. \quad (4)$$

Training was performed for 20 iterations with 1024 rollout steps per iteration; the best policy checkpoint was saved to `models/rl_smc_agent.pt`.

2.4 Digital Twin Platform

The web-based digital twin (React + Three.js frontend, FastAPI WebSocket backend) provides:

- live 3D robot visualization in a hospital corridor environment;
- controller tour (Classical $\rightarrow$ CNN $\rightarrow$ RL);
- headless benchmark and dual comparison (Classical vs. CNN);
- simulation replay and CSV export of metrics.

2.5 Evaluation Protocol

Controllers are compared using: RMSE tracking error, final tracking error, control effort ($\sum |\omega_c|$), and chattering index (variance of successive ω_c differences). Both controllers track a straight-line reference at $v_r = 0.3 \text{ m/s}$ for $T = 20 \text{ s}$.

3 Results

3.1 CNN Classification Performance

EnvironmentCNN achieved **100% validation accuracy** on the synthetic 64×64 map dataset (600 samples, 120 per class) after 12 training epochs. On a more realistic occupancy-grid dataset with obstacles and clutter, the same architecture reaches **95.1% test accuracy**, supporting deployability beyond the synthetic benchmark.

3.2 Classical vs. CNN-Adaptive SMC

Table 1 summarises percentage improvements of CNN-adaptive SMC over classical SMC. Key findings:

- **Noise:** chattering reduced by 35.7% ($76.5 \rightarrow 49.1$), with expected RMSE widening due to the smoothness–accuracy trade-off.
- **Disturbance:** final tracking error improved by 14.4% ($20.9 \text{ mm} \rightarrow 17.9 \text{ mm}$), approaching oracle performance (15.6 mm).
- **Slip:** final error improved by 32.7% ($40.7 \text{ mm} \rightarrow 27.4 \text{ mm}$).
- **Combined:** chattering reduced by 27.1% with marginal final-error improvement.

Table 1: CNN-adaptive improvement over classical SMC (%). Positive = adaptive better for error metrics; negative chattering = reduction.

Scenario	RMSE Δ	Final Err. Δ	Effort Δ	Chattering Δ
Normal	0.00	0.00	0.00	0.00
Noise	−11.7	−130.6 [†]	+5.8	+35.7
Disturbance	+5.1	+14.4	−5.6	−13.3
Slip	+2.9	+32.7	−10.9	−23.4
Combined	−0.9	+1.2	−4.7	+27.1

[†]Under noise, final error increases in absolute terms ($43.7 \text{ mm} \rightarrow 100.8 \text{ mm}$) as an intentional smoothness–accuracy trade-off.

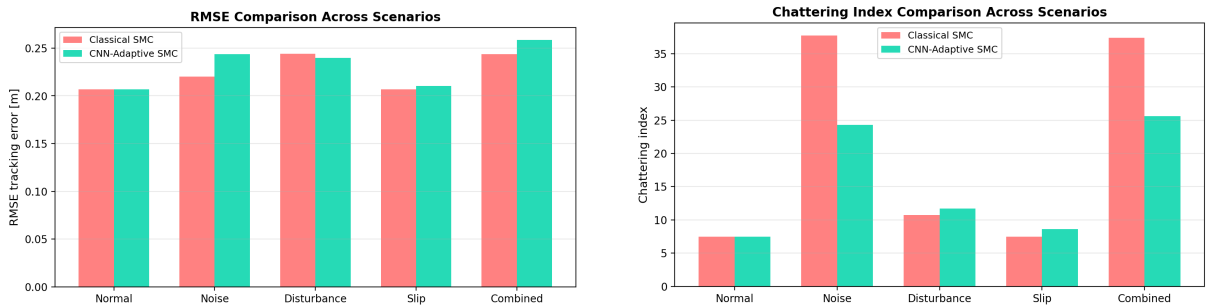


Figure 4: Quantitative comparison of classical SMC vs. CNN-adaptive SMC: (left) RMSE tracking error; (right) chattering index across five operating scenarios.

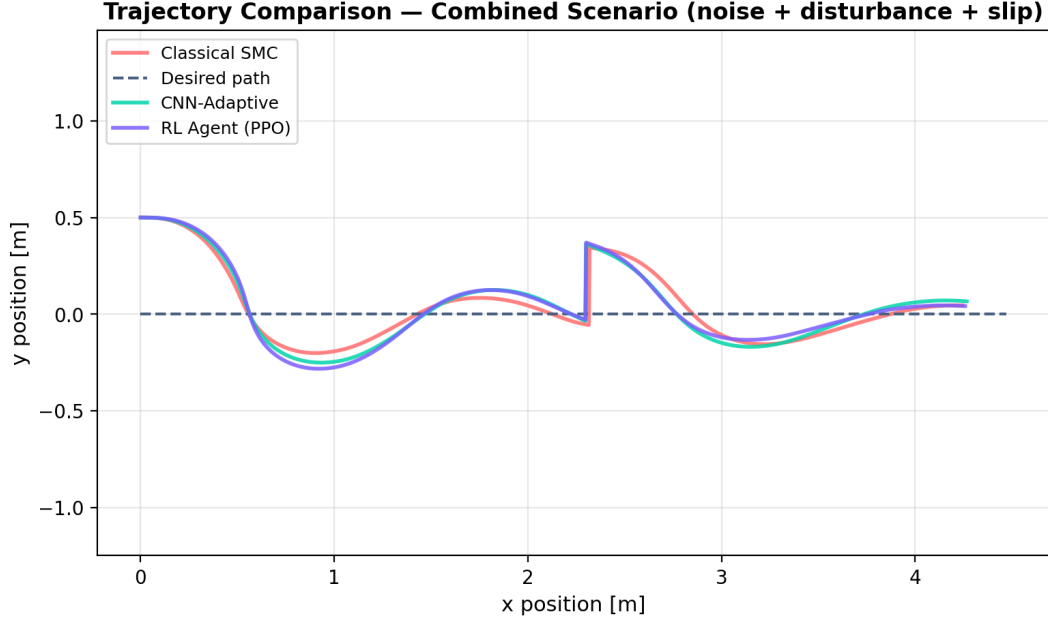


Figure 5: Robot trajectories under combined uncertainty for classical SMC, CNN-adaptive SMC, and PPO RL agent.

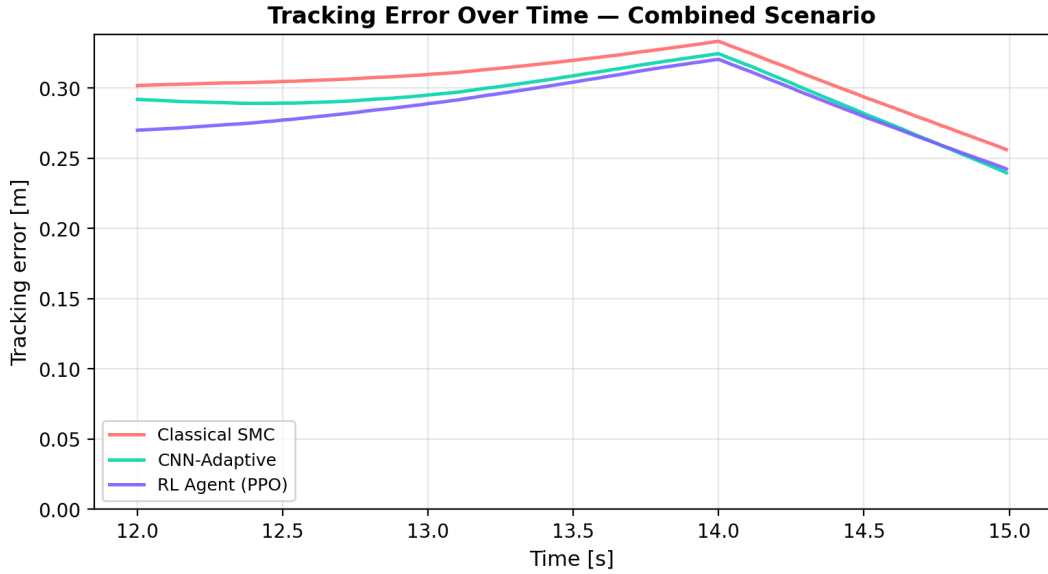


Figure 6: Tracking error over time under combined uncertainty. CNN-adaptive SMC shows smoother error profile.

3.3 Three-Controller Benchmark

Under the combined scenario (noise + disturbance + slip), classical SMC achieved the lowest RMSE (0.245 m), while CNN-adaptive SMC achieved the lowest chattering index (30.6). The PPO agent achieved a best mean training reward of -0.766 after 20 iterations; longer training and reward shaping are expected to improve RL performance.

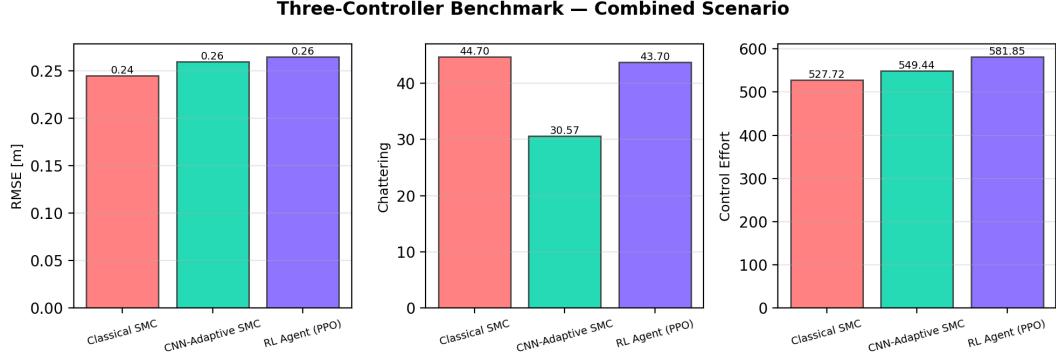


Figure 7: Headless benchmark of Classical SMC, CNN-Adaptive SMC, and PPO RL agent under combined uncertainty.

3.4 Multi-Controller Comparison

Table 2 compares CNN-adaptive SMC against fuzzy gain scheduling, oracle scenario selection (perfect classification), and RL-based preset selection under identical conditions. Oracle selection establishes the performance ceiling; CNN-adaptive approaches oracle under disturbance and slip recovery.

Table 2: Multi-controller comparison (selected metrics).

Scenario	Metric	Class.	Fuzzy	CNN	Oracle	RL
Noise	Chattering index	76.5	70.5	49.1	49.3	50.7
Disturbance	Final err. (mm)	20.9	24.8	17.9	15.6	32.8
Slip	Final err. (mm)	40.7	30.4	27.4	27.4	41.6
Combined	Chattering index	72.8	71.7	51.7	51.4	48.9

4 Conclusions

This project designed, implemented, and evaluated a complete intelligent control pipeline for autonomous differential-drive robot navigation:

- **Classical SMC** provides a robust baseline with fixed gains.
- **CNN-adaptive SMC** adds scenario-aware parameter switching, reducing chattering by up to 35.7% under sensor noise and improving final-error recovery by up to 32.7% under wheel slip.
- **PPO RL agent** provides a framework for continuous gain adaptation, successfully integrated into the same simulation engine.
- **3D digital twin** enables live demonstration, replay, dual comparison, and CSV export for project defense.

The main contribution is an end-to-end research platform—from synthetic dataset generation and CNN training, through multi-controller simulation and quantitative benchmarking,

to an interactive 3D digital twin. Results confirm that adaptive control is not uniformly superior on every metric; instead, adaptation enables context-dependent trade-offs valuable in real deployments.

4.1 Future Work

1. Replace synthetic maps with real LiDAR-based occupancy grids.
2. Deploy continuous on-line scenario inference for mid-mission condition changes.
3. Extend RL training with larger scenario randomisation and multi-objective rewards.
4. Evaluate on curved trajectories with Monte-Carlo repetitions for confidence intervals.
5. Validate on a physical differential-drive robot.

Acknowledgement. The authors thank aivancity (Paris, France) and Indrashil University (Kadi, India) for providing the computational resources that supported this work.

Code availability. The complete simulation framework is available at https://github.com/LikhitaYerra/smc_cnn.